

RAVEL: 面向证据可见性扰动的 Token 高效且写安全的 LLM 多智能体系统

Risk-Adaptive Visibility and Evidence Ledger for Token-Efficient and Write-Safe LLM Multi-Agent Systems

研究 Proposal (中文 LaTeX 版本)

2026 年 6 月 15 日

摘要

大型语言模型驱动的多智能体系统通常通过共享对话历史、工具输出或外部记忆来协调行动。然而，在长程、多工具且会改变外部状态的任务中，系统既面临上下文冗余与 token 成本快速增长，也面临不同智能体看到的工具证据不完整、延迟或相互冲突时产生错误写操作的问题。现有研究分别讨论了上下文压缩、轨迹评估、多智能体故障、共享状态一致性和执行溯源，但尚未充分回答一个更具体的系统问题：在固定任务、模型、工具与通信拓扑的条件下，能否把“谁在何时看到哪些证据”作为显式可优化变量，在显著降低 token 消耗的同时，维持任务效用并提高高风险写操作的证据有效性？

本 proposal 提出 RAVEL (Risk-Adaptive Visibility and Evidence Ledger)。该框架将完整工具输出保存在外部、版本化且可追溯的证据账本中；常规回合仅向相关智能体分发最小充分的 header、delta 与 pointer；当系统提出退款、下单、改签、账户变更或配置修改等高风险写操作时，再由提交门控器依据动作模式拉取必要字段、检测陈旧与冲突、触发选择性重查，并按照风险自适应地分配和解与推理预算。研究将基于 τ -bench 的 airline/retail 与 τ^2 -bench 的 telecom 环境构建严格配对的可见性干预，保留原 benchmark 的任务、策略与数据库语义，只通过可复现的 wrapper 注入 FULLSYNC、DELAYED、FIELDMASK 与 CONFLICTINGVIEW 四类 observation regimes。实验采用 gpt-oss-120b、Qwen3-32B、GLM-4.5-Air 和 Llama-3.3-70B-Instruct，在固定编排架构内比较全量同步、共享 scratchpad、通用 observation masking、ledger-only、commit-gate-only 与 RAVEL 的性能。

核心评价不只包括最终任务成功率，还包括未缓存 prompt token、轨迹分叉位置、工具顺序与参数正确性、陈旧写率、冲突写率、不安全写率、冲突拦截率、恢复率和过度拦截率。统计上使用任务级配对 bootstrap、混合效应模型及预注册的非劣效边界。预期贡献是：将 evidence visibility 从潜在实现细节提升为一等算法变量；给出 token-utility-write-safety 的统一形式化；提出可落地的 ledger、最小证据路由、两阶段提交与风险自适应和解机制；并建立一个能够隔离模型能力、架构拓扑和可见性扰动的严格评估协议。

关键词： LLM 多智能体；工具调用；证据可见性；上下文工程；写安全；执行溯源；共享状态一致性；Token 效率

目录

1 研究背景与动机	4
1.1 从“多智能体协作”转向“证据如何进入每个智能体的上下文”	4

目录	2
1.2 为什么“全部同步”不是显然正确的答案	4
1.3 核心研究缺口	5
2 研究问题、假设与预期贡献	5
2.1 研究问题	5
2.2 可证伪假设	5
2.3 预期贡献	6
3 问题形式化	6
3.1 多智能体工具环境	6
3.2 证据可见性的四个维度	7
3.3 高风险写与证据有效性	7
3.4 联合优化目标	7
4 方法：RAVEL	8
4.1 总体架构	8
4.2 模块一：可验证增量证据账本（VDL）	8
4.3 模块二：最小充分证据路由（MSE-Router）	9
4.4 模块三：两阶段写提交与证据门控	9
4.5 模块四：风险自适应和解预算（ARB）	10
4.6 条件化安全性质与复杂度分析	11
5 实验设计	11
5.1 总体实验逻辑：先证实现象，再验证方法	11
5.2 Benchmark 选择与改造原则	12
5.3 Observation regimes	13
5.4 模型与部署	13
5.5 多智能体架构控制	13
5.6 基线与方法变体	14
5.7 样本与计算预算	14
6 评价指标与统计分析	15
6.1 任务与可靠性指标	15
6.2 Token 与效率指标	15
6.3 写安全指标	15
6.4 轨迹指标	16
6.5 统计检验	16
6.6 预注册成功标准	16
7 消融实验、混淆控制与稳健性分析	17
7.1 核心消融	17
7.2 主要混淆因素与控制	17
7.3 稳健性与负面结果分析	18

目录	3
8 预期结果与论文叙事	19
8.1 预期但不预设的结果模式	19
8.2 论文主图与主表	19
8.3 可接受的 claim 边界	19
9 工程实施计划	20
9.1 阶段与交付物	20
9.2 软件与日志要求	20
10 风险、伦理与局限	21
10.1 方法风险	21
10.2 安全与伦理边界	21
10.3 开放科学与复现	21
11 结论	21

1 研究背景与动机

1.1 从“多智能体协作”转向“证据如何进入每个智能体的上下文”

工具增强型语言智能体通过外部 API、数据库、检索器和执行环境完成单纯语言生成无法完成的任务。ReAct 将推理与行动交替组织为可执行轨迹，为后续 agentic tool use 奠定了基础 [Yao et al., 2023]; AutoGen 等框架进一步把多个可对话、可调用工具的智能体组织为协作系统 [Wu et al., 2023]。然而，多智能体系统的新增能力同时引入了新的信息流问题：一个工具结果可能被某个 worker 完整看到、被 supervisor 只看到摘要、被另一个 agent 延迟看到，或以不同版本出现在共享 memory 中。此时，系统的全局状态并不等价于任何单一智能体的局部上下文。

已有 benchmark 已经表明，真实 agent 任务的难点不只在于“会不会调用某个函数”。 τ -bench 要求智能体在 airline 和 retail 场景中同时遵守政策、与用户交互并改变数据库状态，并以最终数据库状态而非文本答案评估成功 [Yao et al., 2024]。 τ^2 -bench 将环境扩展到 agent 与 user 都能通过工具改变共享世界的 dual-control 场景，显著增加了协调难度 [Barres et al., 2025]。DICE-Bench 进一步指出，工具名和参数等关键信息在多轮、多方对话中的分散程度本身会影响函数调用能力 [Jang et al., 2025]; TRAJECT-Bench 则强调必须检查工具选择、参数、依赖顺序与完整轨迹，而不能只看最终答案 [He et al., 2025]。

这些工作共同提示：**工具证据的范围、粒度、时序和版本会改变 agent 的决策轨迹**。本项目已有先锋观察也指向同一现象：在其余系统条件保持不变时，仅改变 agent 可见的工具证据，就可能导致后续 tool trajectory 分叉。但该观察仍不足以直接支持强因果或算法性结论，因为此前实验可能同时混入任务难度、模型随机性、架构差异、摘要质量、缓存和环境状态变化等混淆因素。因此，本 proposal 的第一原则不是直接宣称“可见性必然造成失败”，而是通过严格配对干预，把 visibility 本身从其他变量中拆出来。

1.2 为什么“全部同步”不是显然正确的答案

一种直觉做法是让所有智能体持续看到所有原始工具输出，即 full synchronization。它能减少显式信息缺失，却带来三类问题。

1. **成本与上下文拥塞**。多智能体系统会重复处理相同工具输出。长轨迹中，raw payload、历史计划与多 agent 消息被反复拼接，token 成本近似随 agent 数和回合数共同增长。针对长程 agent 的研究已经发现，简单 observation masking、依赖结构化记忆或自适应省略冗余观察能够在不明显损害任务效用的情况下显著降低上下文成本 [Lindenbauer et al., 2025, Wu et al., 2026, Ning et al., 2026]。
2. **无关与冲突信息扩散**。全量广播不仅传播有用信息，也传播过期值、重复消息、未经验证的摘要和与当前角色无关的细节。MAST 对多智能体失败的系统分析将失败归纳为系统设计、智能体间失配以及任务验证/终止问题 [Cemri et al., 2025]; MAS-FIRE 进一步显示，消息路由和架构拓扑本身会显著影响故障传播与恢复 [Jia et al., 2026]。
3. **写操作缺少证据边界**。即使上下文很长，模型仍可能无法明确回答“本次写入依赖的是哪个版本、哪些字段、由哪个工具产生”。共享状态研究和执行溯源研究分别指出，陈旧读取、并发冲突和缺失 provenance 会使系统无法可靠判断一次状态改变是否被有效证据支持 [Khan, 2026, Wang et al., 2026]。

因此，问题不应被表述为“是否同步”，而应被表述为：**在什么任务阶段、对哪个 agent、以**

什么粒度同步哪些证据，以及在写入之前如何补足并验证读依赖。

1.3 核心研究缺口

现有工作可分为五条相邻但未完全重合的路线：

- **工具与轨迹 benchmark:** τ -bench、 τ^2 -bench、DICE-Bench、TRAJECT-Bench 主要回答如何构造和评估真实、多轮、多工具任务 [Yao et al., 2024, Barres et al., 2025, Jang et al., 2025, He et al., 2025]。
- **多智能体故障与协调:** MAST 和 MAS-FIRE 主要识别失败类型、故障传播和拓扑鲁棒性 [Cemri et al., 2025, Jia et al., 2026]。
- **上下文效率:** observation masking、Agent-Omit、ContextWeaver 等主要优化单 agent 或通用 agent 的上下文保留策略 [Lindenbauer et al., 2025, Ning et al., 2026, Wu et al., 2026]。
- **共享状态一致性与执行 provenance:** S-Bus、Agent-Sentry 及近期 provenance survey 关注 read-set、状态冲突、参数来源和审计链 [Khan, 2026, Sequeira et al., 2026, Wang et al., 2026]。
- **工具权限与安全门控:** AgentDojo 和 Progent 等研究工具环境中的 prompt injection、权限边界与危险调用控制 [Debenedetti et al., 2024, Shi et al., 2025]。

本 proposal 聚焦的交叉缺口是：把多智能体内部 evidence visibility policy 形式化为可优化的运行时策略，并在高风险写操作上联合优化 token 成本、任务效用和证据有效性。这里的目标既不是仅压缩上下文，也不是仅阻止未经授权的工具调用，更不是重新发明数据库事务；而是把 prompt 可见信息、外部证据状态、动作依赖和提交安全连接成一个统一闭环。

2 研究问题、假设与预期贡献

2.1 研究问题

RQ1: 可见性因果效应。 在固定任务、工具、环境初态、模型、prompt、解码参数和通信拓扑时，仅改变证据可见性 regime，是否会系统性改变工具轨迹、最终状态与写安全？

RQ2: 效率-效用权衡。 与 full synchronization 和共享 scratchpad 相比，最小充分证据路由是否能显著减少未缓存 prompt token，同时保持最终任务成功率非劣？

RQ3: 写安全。 外部证据账本与提交门控是否能降低 stale、conflicting 和 unsupported writes，并且不会仅通过过度阻断换取表面安全？

RQ4: 风险自适应预算。 只在高风险写、冲突或证据陈旧时触发更深推理和选择性重查，能否优于“所有回合都使用高推理预算”或“所有回合都使用低预算”？

RQ5: 稳健性与外推。 上述效果能否跨模型、任务域、可见性扰动强度及次级架构复现？

2.2 可证伪假设

- **H1 (轨迹敏感性):** 相对于 FULLSYNC, DELAYED, FIELDMASK 与 CONFLICTINGVIEW 将显著提高轨迹分叉率，并在高风险写切片上降低证据有效性。

- **H2 (token 非劣效):** RAVEL 相比 multi-agent full-sync 至少降低 20% 的未缓存总 prompt token, 同时任务成功率的绝对下降不超过预注册非劣效边界 $\epsilon = 2$ 个百分点。
- **H3 (安全优越性):** 在 delayed/conflicting regimes 下, RAVEL 相比 full-sync、ledger-only 与 commit-gate-only 显著降低 Unsafe Action Rate (UAR) 和 Conflicting Write Rate (CWR)。
- **H4 (非过度阻断):** RAVEL 的安全收益不能由无限制 abstention 解释; 其 Overblock Rate 不高于 12%, 且被拦截 trial 中至少 50% 能通过选择性和解恢复为正确终态。
- **H5 (组件必要性):** 移除最小证据路由、版本化 ledger、commit gate 或 adaptive reconciliation 中任一核心组件, 都会在 token、write safety 或 recovery 中至少损害一个主要结果。

2.3 预期贡献

1. **问题贡献:** 将 evidence visibility policy 定义为决定多智能体 tool trajectory 的一等变量, 而不是实现层面的偶然差异。
2. **方法贡献:** 提出 RAVEL, 将最小充分证据路由、可验证增量账本、两阶段写提交和风险自适应和解统一到一个运行时框架中。
3. **评估贡献:** 基于现有公开 benchmark 构造严格配对的 observation-regime perturbation protocol, 同时评价 final state、trajectory、token 与 write safety。
4. **实证贡献:** 系统刻画 visibility 的影响在不同模型、领域、架构和风险级别下何时成立、何时失效, 并给出 token-utility-safety Pareto frontier。

3 问题形式化

3.1 多智能体工具环境

将系统建模为带共享环境状态的部分可观测多智能体决策过程:

$$\mathcal{M} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{O}_i\}_{i=1}^N, \{\mathcal{A}_i\}_{i=1}^N, \mathcal{T}, \mathcal{G}, \mathcal{P} \rangle, \quad (1)$$

其中 $\mathcal{N} = \{1, \dots, N\}$ 为智能体集合, $s_t \in \mathcal{S}$ 为环境在时刻 t 的真实状态, $a_t^i \in \mathcal{A}_i$ 为智能体 i 的语言、委派或工具动作, $\mathcal{T}$ 为环境转移函数, $\mathcal{G}$ 为任务目标, $\mathcal{P}$ 为领域政策与约束。

每次工具调用产生原始证据单元

$$e_j = \langle \text{object}_j, \text{tool}_j, \text{payload}_j, \text{version}_j, \text{time}_j, \text{source}_j, \text{digest}_j \rangle. \quad (2)$$

证据并不自动进入所有 agent 的上下文。可见性策略 V_ϕ 根据历史、角色、子目标和风险, 为每个 agent 生成局部视图:

$$o_t^i = V_\phi(i, g_t^i, H_t, \mathcal{L}_t, r_t), \quad (3)$$

其中 H_t 是交互历史, $\mathcal{L}_t$ 是外部 evidence ledger, g_t^i 是局部子目标, r_t 是当前风险状态。于是完整工具轨迹为

$$\tau = (o_1, a_1, e_1, \dots, o_T, a_T, e_T, s_{T+1}). \quad (4)$$

本研究的关键干预变量不是 s_t 本身, 而是 V_ϕ 输出给不同 agent 的证据投影。

3.2 证据可见性的四个维度

对于证据集合 E_t ，可见性由四个互相独立但可组合的维度描述：

$$V = \langle \text{scope}, \text{granularity}, \text{timing}, \text{version} \rangle. \quad (5)$$

- **Scope**: 哪些 agent 可以看到该证据；
- **Granularity**: raw payload、字段切片、摘要、delta 还是 pointer；
- **Timing**: 立即、延迟若干 agent event，或只在写前按需拉取；
- **Version**: 当前版本、历史版本或存在冲突的多个版本。

3.3 高风险写与证据有效性

令 w_t 表示会改变外部持久状态的候选写动作，例如退款、预订、改签、地址变更、账户设置或网络配置修改。每个动作类型 a 对应一个预定义的 required evidence schema：

$$\mathcal{R}(a) = \{f_1, f_2, \dots, f_K\}, \quad (6)$$

其中每个 f_k 是提交前必须读取并验证的字段或约束。

定义动作 w_t 的证据有效性为

$$\begin{aligned} \text{EVIDENCEVALID}(w_t) = & \text{Complete}(w_t) \wedge \text{Fresh}(w_t) \wedge \\ & \text{ConflictFree}(w_t) \wedge \text{Traceable}(w_t) \wedge \text{PolicyCompliant}(w_t). \end{aligned} \quad (7)$$

五个分量分别表示：必需字段齐全；字段版本未被更新覆盖；不存在未解决冲突；关键参数能回溯到原始工具证据；动作满足领域政策。

3.4 联合优化目标

令总 token 成本为

$$C_{\text{tok}}(\tau) = \sum_{t=1}^T \sum_{i=1}^N \left(\text{Tok}_t^{i,\text{in}} + \text{Tok}_t^{i,\text{out}} \right), \quad (8)$$

并单独记录未缓存 prompt token，避免 provider caching 造成伪节约。令 J_{task} 表示最终数据库状态与目标状态的一致性，令 L_{safe} 汇总 stale、conflicting、unsupported writes 和过度阻断。研究目标不是单一最小化 token，而是求解约束优化：

$$\begin{aligned} \min_{\pi, V_\phi} \quad & \mathbb{E}[C_{\text{tok}}(\tau) + \lambda_r C_{\text{recon}}(\tau)] \\ \text{s.t.} \quad & \mathbb{E}[J_{\text{task}}(\tau)] \geq J_{\text{base}} - \epsilon, \\ & \Pr(\text{EVIDENCEVALID}(w) = 1 \mid w \text{ executed}) \geq 1 - \delta, \\ & \text{Overblock} \leq \eta. \end{aligned} \quad (9)$$

其中 ϵ 为任务非劣效边界， δ 为允许的不安全写上限， η 为允许的过度阻断率。实验主张将依据 Pareto frontier，而不是用任意加权和掩盖安全与效用之间的冲突。

4 方法: RAVEL

4.1 总体架构

主实验采用固定的 orchestrator-supervisor 拓扑, 以避免把架构差异误当作 visibility 效应。系统包含四类角色:

1. **Supervisor**: 分解任务、维护全局子目标并决定委派;
2. **Policy Agent**: 检索并解释领域政策、动作前置条件和 required evidence schema;
3. **Tool Worker**: 执行只读工具、整合局部结果并提出候选写;
4. **Commit Verifier**: 独立检查证据有效性, 决定 commit、reconcile、replan 或 abstain。

在同一实验 run 中, 四个角色使用同一基础模型与固定 prompt 模板, 避免因异构模型造成能力混淆。只有 verifier 可以真正触发高风险写工具; worker 只能生成候选动作。

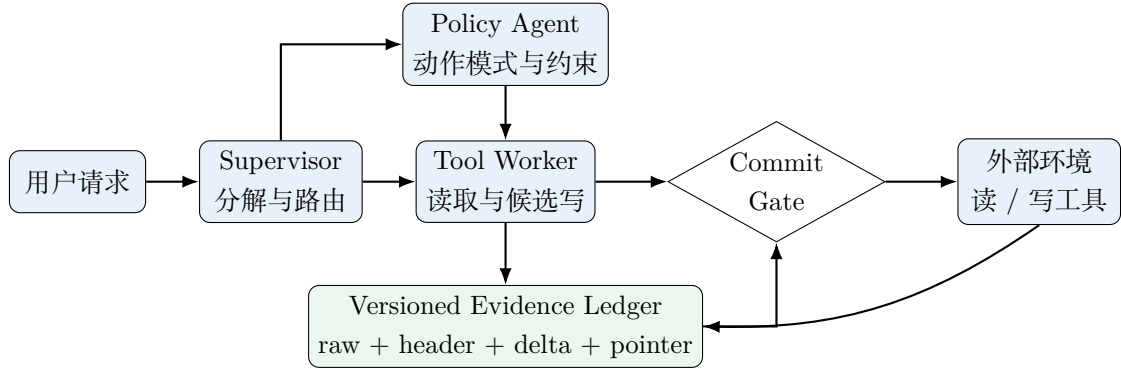


图 1: RAVEL 总体架构。常规回合只传播最小证据表示; 高风险写在提交前依据 ledger 中的版本化证据进行验证。

4.2 模块一: 可验证增量证据账本 (VDL)

VDL (Verifiable Delta Ledger) 把“证据存储”与“prompt 可见内容”分离。每条工具结果被规范化为 ledger entry:

$$\ell_j = \langle id, object, tool, version, time, source, raw, delta, digest, deps \rangle. \quad (10)$$

其中 raw payload 不默认进入任何 agent 的 prompt; 系统通常只分发 compact header:

$$h_j = \langle object, version, changed_fields, time, source, risk, conflict, digest, pointer \rangle. \quad (11)$$

VDL 的设计原则如下:

- **不可变追加**: 新工具结果形成新版本, 不原地覆盖旧记录;
- **字段级 delta**: 只显式传播相对前一版本发生变化的字段;
- **来源与依赖**: 记录证据来自哪个工具、被哪些候选动作引用;
- **稳定摘要**: digest 用于检测同一对象同一版本是否被篡改或错误复用;

- **原文可回取:** 当摘要、字段切片或冲突检测不足时, 可通过 pointer 拉取 raw payload。

该模块不声称解决所有语义 provenance。它优先覆盖结构化或半结构化工具输出; 对于自由文本, 只提供 chunk pointer、短 excerpt 与来源信息, 避免把不可靠自动摘要误当作完整证据。

4.3 模块二: 最小充分证据路由 (MSE-Router)

MSE-Router 根据角色、当前子目标和潜在动作, 只向 agent 提供执行该局部决策所需的证据。对 agent i 在时刻 t , 路由器选择字段集合

$$F_t^i = \arg \min_{F \subseteq E_t} \text{Tok}(F) \quad \text{s.t.} \quad \text{Sufficient}(F, g_t^i, \mathcal{P}) = 1. \quad (12)$$

第一阶段采用透明、可审计的 rule-based schema, 而不立即训练 learned router:

$$F_t^i = F_{\text{role}(i)} \cap F_{g_t^i} \cup F_{\text{dependencies}(g_t^i)}. \quad (13)$$

对只读或低风险步骤, 系统发送 header、delta preview 和必要字段; 对高风险候选写, 系统不相信 agent 当前上下文“恰好完整”, 而由 commit gate 重新依据动作 schema 构造最小 read-set。

这种设计避免两个极端: 一是所有 agent 始终重放完整 raw outputs; 二是仅依赖不可审计的 LLM 摘要。路由错误的保守 fallback 是扩大字段集合或拉取 raw payload, 而不是直接提交。

4.4 模块三: 两阶段写提交与证据门控

所有状态改变动作采用 propose-validate-commit 两阶段流程:

1. Tool Worker 生成结构化候选写 $\hat{w}_t$, 包括动作名、参数、目标对象和引用的 evidence IDs;
2. Commit Verifier 从 $\mathcal{R}(\hat{w}_t)$ 构造 required read-set;
3. ledger 返回最新字段、版本、来源、冲突标记与政策依据;
4. 若满足 EVIDENCEVALID, 则执行真实写工具; 否则进入 reconciliation ladder;
5. 写工具返回的新状态再次写入 ledger, 并形成新版本。

Algorithm 1 RAVEL 的风险自适应执行流程**Require:** 用户目标 G , 领域政策 $\mathcal{P}$, 工具集合 $\mathcal{U}$, ledger $\mathcal{L}$

```

1: 初始化 supervisor plan  $P_0$  与风险状态  $r_0$ 
2: for  $t = 1, \dots, T_{\max}$  do
3:    $i, g_t^i \leftarrow \text{DELEGATE}(P_{t-1}, \mathcal{L})$ 
4:    $o_t^i \leftarrow \text{ROUTE MINIMAL EVIDENCE}(i, g_t^i, \mathcal{L}, r_t)$ 
5:    $a_t^i \leftarrow \text{AGENT ACT}(o_t^i, g_t^i)$ 
6:   if  $a_t^i$  是读操作或低风险可逆动作 then
7:      $e_t \leftarrow \text{EXECUTE}(a_t^i)$ 
8:      $\mathcal{L} \leftarrow \text{APPEND VERSIONED EVIDENCE}(\mathcal{L}, e_t)$ 
9:   else if  $a_t^i$  是候选高风险写  $\hat{w}_t$  then
10:     $R_t \leftarrow \mathcal{R}(\hat{w}_t)$ 
11:     $E_t^{\text{req}} \leftarrow \text{FETCH LATEST}(\mathcal{L}, R_t)$ 
12:    if  $\text{EVIDENCE VALID}(\hat{w}_t; E_t^{\text{req}}, \mathcal{P})$  then
13:       $e_t \leftarrow \text{COMMIT}(\hat{w}_t, E_t^{\text{req}})$ 
14:       $\mathcal{L} \leftarrow \text{APPEND VERSIONED EVIDENCE}(\mathcal{L}, e_t)$ 
15:    else
16:       $E'_t \leftarrow \text{ADAPTIVE RECONCILE}(\hat{w}_t, E_t^{\text{req}}, \mathcal{L})$ 
17:      if  $\text{EVIDENCE VALID}(\hat{w}_t; E'_t, \mathcal{P})$  then
18:         $e_t \leftarrow \text{COMMIT}(\hat{w}_t, E'_t)$ 
19:      else
20:         $P_t \leftarrow \text{REPLAN OR ABSTAIN}(P_{t-1}, E'_t)$ 
21:      end if
22:    end if
23:  end if
24:  if 终态满足目标或达到安全终止条件 then
25:    return trajectory  $\tau$  与 final state
26:  end if
27: end for

```

4.5 模块四: 风险自适应和解预算 (ARB)

ARB (Adaptive Reconciliation Budget) 只在风险上升时增加上下文、工具重查和推理预算。风险评分由可审计特征计算:

$$r_t = \alpha_1 \mathbb{I}[\text{irreversible}] + \alpha_2 \mathbb{I}[\text{stale}] + \alpha_3 \mathbb{I}[\text{conflict}] + \alpha_4 \mathbb{I}[\text{missing fields}] + \alpha_5 \mathbb{I}[\text{policy ambiguity}]. \quad (14)$$

和解阶梯依次为:

1. 拉取缺失字段;
2. 拉取相关对象的最新 delta;
3. 拉取冲突版本的 raw payload;
4. 选择性重新调用产生冲突字段的读取工具;

5. 提高 reasoning budget 或调用独立 verifier;
6. 若仍无法建立有效证据, 则 replan、请求用户确认或安全 abstain。

不同模型的 thinking/reasoning 开关并不等价, 因此实验不会把跨模型 budget 数值直接比较。系统通过统一抽象层将其映射为 low/normal/high 三档, 并在每个模型内部比较固定预算和风险自适应预算。gpt-oss、Qwen3 和 GLM-4.5-Air 均公开支持不同形式的推理模式或预算控制 [OpenAI, 2025, Qwen Team, 2025, Z.ai, 2025]; Llama-3.3-70B-Instruct 作为无显式 hybrid-thinking 开关的对照, 通过最大生成 token 和 verifier 调用次数实现预算档位 [Meta AI, 2024]。

4.6 条件化安全性质与复杂度分析

命题 1 (条件化提交安全)。 若满足以下假设: (i) 每类高风险动作的 required evidence schema 完整; (ii) ledger 版本单调且最新读取正确; (iii) 冲突检测对结构化字段无漏报; (iv) 真实写工具只能由 commit gate 调用; (v) commit 检查与写入之间不存在未检测的外部竞争更新, 则 RAVEL 不会提交 EVIDENCEVALID = 0 的结构化高风险写。

证明思路。 任一高风险写必须先进入 gate。gate 按完整 schema 构造 read-set, 并在假设 (ii)–(iii) 下正确判断 freshness 与 conflict; 假设 (iv) 排除了绕过 gate 的写; 假设 (v) 保证验证时版本与提交时版本一致。因此, 不满足任一有效性条件的候选写只能进入 reconcile/replan/abstain, 而不能 commit。该性质是条件化的系统保证, 不覆盖 schema 遗漏、自由文本语义冲突或外部系统绕过版本控制的情况。

命题 2 (证据传输成本上界)。 设 N 为 agent 数, T 为决策轮数, B 为平均 raw tool output 长度, h 为 compact header/delta 长度, W 为候选高风险写数, k 为每次写所需字段切片的平均 token, Q 为需要 raw fetch 的和解次数。naive full-sync 的证据重放成本为 $\Theta(NTB)$; RAVEL 的证据传输成本上界为

$$O(NTh + Wk + QB). \quad (15)$$

当 $h \ll B$ 、 $W \ll NT$ 且 $Q \ll NT$ 时, RAVEL 可获得严格的渐近节约。若所有回合都触发冲突和 raw fetch, 则该优势可能消失, 因此冲突密度将被单独报告。

5 实验设计

5.1 总体实验逻辑: 先证实现象, 再验证方法

实验被分成两个互相区分的阶段, 避免把“可见性确实有影响”与“所提方法优于基线”混在一起。

1. **Stage A: Controlled Visibility Perturbation。** 固定同一 multi-agent architecture 和同一基础方法, 只改变 observation regime, 估计可见性对 trajectory、final state 和 write safety 的影响。
2. **Stage B: Method Evaluation。** 在相同任务与 perturbation 下比较 full-sync、masking、ledger、gate 和 RAVEL, 检验 token–utility–safety 权衡。

5.2 Benchmark 选择与改造原则

主任务不从零构造，而是从公开、可执行的 benchmark 中筛选和受控改造：

表 1: 实验任务域与用途

Benchmark/域	角色	采用理由	本研究改造方式
τ -bench Airline	主域	多轮用户交互、政策约束、读写 API、可验证数据库终态	仅筛选长程高风险任务；通过 wrapper 控制证据投影与版本，不改任务目标和政策
τ -bench Retail	主域	订单、退款、换货、支付状态等典型高风险写	同上；注入受控的订单/库存/支付状态中途变化
τ^2 -bench com	Tele- 边界域	agent 与 user 都能通过工具改变共享环境，天然包含 coordination	保留 dual-control 语义；控制 agent 侧证据延迟、字段遮蔽与版本冲突
AgentDojo 子集	可选安全外推	包含 e-banking、travel、email 等工具任务和不可信数据	只做安全 transfer，不作为主结果；不引入攻击时评价正常 utility，有攻击时评价 provenance/gate 的附加作用

任务进入主评估前必须满足：

1. 经环境依赖图、程序化 oracle 或双人标注确认，合理成功路径至少需要 5 次工具调用；
2. 至少包含 3 层依赖，即后续工具参数或决策依赖先前工具结果；
3. 至少包含一次会改变外部持久状态的候选写；
4. 至少有一个预注册 checkpoint，可在不改变用户最终目标的前提下注入环境状态变化，迫使 agent 根据工具结果更新计划；
5. 任务在未注入扰动时对至少一个基线模型可解，避免把完全不可解任务误当 visibility failure。

中途环境变化不是重新编写任务，而是 benchmark wrapper 对原有数据库状态执行合法、可记录的 exogenous mutation，例如价格/库存/座位可用性变化、订单状态推进、支付状态更新或用户侧 modem 配置变化。mutation 的时刻、字段和新版本在 task seed 中预先固定。

5.3 Observation regimes

表 2: 证据可见性干预

Regime	操作化定义	目的
FULLSYNC	所有相关 agent 在同一逻辑时刻收到相同完整 raw tool output	高成本、强同步参照
DELAYED	指定 agent 延迟 $d \in \{1, 2\}$ 个 agent event 才收到最新版本；其余内容不变	隔离 freshness 与传播时序
FIELDMASK	对非证据所有者隐藏一个或多个 action-critical 字段，但 ledger/raw state 保持不变	隔离 granularity 与信息缺失
CONFLICTINGVIEW	不同 agent 分别收到同一对象的历史版本 v 与当前版本 $v + 1$ ，或两个互斥字段值，并保留版本标识	测试冲突检测与写前和解

所有 regime 使用相同 task seed、数据库初态、exogenous mutation、user simulator seed、agent prompt、工具实现和 decoding 设置。扰动由中间件确定性执行，不让额外 LLM 生成，以免引入语义质量差异。

5.4 模型与部署

模型选择覆盖不同结构、规模与推理范式：

- **gpt-oss-120b**: open-weight reasoning model，作为强推理主模型；
- **Qwen3-32B**: dense 32B 模型，支持 thinking/non-thinking 切换；
- **GLM-4.5-Air**: 面向 agent 应用的 MoE 模型，支持 thinking/non-thinking；
- **Llama-3.3-70B-Instruct**: 70B instruction-tuned dense model，作为成熟通用开源对照。

部署在 4×NVIDIA A100 80GB 集群。每次只加载一个模型，使用统一 OpenAI-compatible serving interface 和固定 tool-call parser。不同模型可使用其官方支持的原生精度/量化格式，但所有方法比较均在**同一模型、同一 checkpoint、同一 serving 配置**内部完成，不把跨模型绝对差异解释为方法因果效应。温度默认设为 0；考虑 GPU kernel 与模型内部仍可能存在非完全确定性，每个 task-regime-method 组合至少重复 3 次，并固定可控制的随机种子。

5.5 多智能体架构控制

主实验只使用 figure 1 的 orchestrator-supervisor 架构。次级架构外推包括：

- **Pipeline/Handoff**: 信息按固定顺序传递；
- **Blackboard**: agent 从共享 memory 读取并写入消息；

次级实验仅回答“方法是否跨拓扑复现”，不与主实验混合估计。所有架构中的 agent 数、角色职责、可用工具和最大调用预算尽量对齐。

5.6 基线与方法变体

表 3: 主要比较方法

方法	证据管理	写控制
Single-Agent ReAct	单 agent 全历史	模型直接调用工具
MAS-FullSync	所有 agent 接收完整 raw outputs	worker 可直接写或统一按原框架设置
Shared Scratchpad	所有 agent 读取累积文本 memory	无版本化证据检查
Observation Masking	固定窗口或删除旧 observation	无专用 write evidence gate
Ledger-Only	raw 在 ledger, prompt 接收 header/pointer	无 commit gate
Commit-Gate-Only	全量上下文	写前检查, 但不做最小路由与 delta
RAVEL-MSE	最小充分字段路由	schema-based commit gate
RAVEL-Full	MSE + VDL + provenance + adaptive reconciliation	完整两阶段提交

5.7 样本与计算预算

采用分层计算设计:

1. **开发集:** 每个主域 30 个 task seed, 用于修复 schema、日志与工具兼容, 不进入最终显著性检验;
2. **Stage A 主评估:** airline 与 retail 各 100 个 paired tasks, telecom 60 个 paired tasks; 四模型、四 regimes、三次重复;
3. **Stage B 核心矩阵:** 所有四模型运行 MAS-FullSync、Commit-Gate-Only 与 RAVEL-Full;
4. **扩展基线与完整消融:** 在一个强模型和一个中等模型上运行全部基线与组件消融;
5. **最终样本调整:** 根据开发集的配对差异方差进行 simulation-based power analysis; 若某域有效样本不足或成本过高, 保持任务级预注册最小样本并透明报告。

这种分层避免在所有模型上运行过大的完整笛卡尔积, 同时保证核心 claim 在四个模型上复现。

6 评价指标与统计分析

6.1 任务与可靠性指标

Final State Success (FSS)。依据 benchmark 官方 evaluator, 比较最终数据库状态与目标状态。对于 τ -bench 同时报告 pass^k , 衡量重复执行的一致成功概率 [Yao et al., 2024]。

Policy Compliance。动作是否违反领域政策、前置条件或用户授权。政策违规与终态失败分开报告, 因为某些错误动作可能偶然得到正确终态。

6.2 Token 与效率指标

$$\text{Tokens}_{\text{total}} = \sum (\text{input} + \text{output}), \quad (16)$$

$$\text{Tokens}_{\text{uncached}} = \sum \text{uncached input tokens}, \quad (17)$$

$$\text{Tokens}_{\text{write}} = \sum_{t \in \text{candidate-write window}} \text{tokens}_t. \quad (18)$$

同时报告 tool calls、LLM calls、wall-clock latency、ledger fetch 次数、raw fetch 次数和 reconciliation steps。主要成本 claim 基于 tokenizer-level uncached token, 而不是受 provider cache 影响的账单值。

6.3 写安全指标

令 W 为执行的写集合, $\hat{W}$ 为所有候选写集合:

$$\text{EvidenceValidRate} = \frac{\sum_{w \in W} \mathbb{I}[\text{EVIDENCEVALID}(w) = 1]}{|W|}, \quad (19)$$

$$\text{SAR} = \frac{\#\text{stale commits}}{|W|}, \quad (20)$$

$$\text{CWR} = \frac{\#\text{conflicting commits}}{|W|}, \quad (21)$$

$$\text{UAR} = \frac{\#\text{commits with EVIDENCEVALID} = 0}{|W|}, \quad (22)$$

$$\text{CWCR} = \frac{\#\text{caught conflicting candidates}}{\#\text{oracle conflicting candidates}}, \quad (23)$$

$$\text{Recovery} = \frac{\#\text{initially invalid trials recovered to correct final state}}{\#\text{initially invalid trials}}, \quad (24)$$

$$\text{Overblock} = \frac{\#\text{blocked candidates judged safe and necessary by oracle}}{\#\text{blocked candidates}}. \quad (25)$$

若分母为零, 该指标记为 not applicable, 并使用 task-level hierarchical model, 而不把零分母强行记为零。

6.4 轨迹指标

将轨迹规范化为工具动作序列

$$\bar{\tau} = [(u_1, \theta_1), (u_2, \theta_2), \dots, (u_m, \theta_m)], \quad (26)$$

其中 u_j 是 canonical tool name, θ_j 是标准化参数。报告:

- 归一化 trajectory edit distance;
- 第一次分叉的 time-to-divergence;
- tool selection accuracy、argument accuracy;
- dependency/order satisfaction;
- 无效重试、循环与不必要调用比例;
- 从错误候选动作到恢复的路径长度。

这些指标与 trajectory-aware evaluation 的思想一致 [He et al., 2025], 但本研究额外关联每一步可见证据与最终写安全。

6.5 统计检验

1. **配对分析:** 对同一 task seed 在不同 regime/method 下的差异做 paired bootstrap, 按 task 聚类, 给出 95% CI;
2. **二元结局:** FSS、EvidenceValid、UAR 使用 mixed-effects logistic regression;
3. **计数与连续结局:** token、tool calls、reconciliation 使用 Gamma/log-normal 或负二项 mixed model, 根据残差与过度离散选择;
4. **固定效应:** method、regime、model、domain 及预注册交互项;
5. **随机效应:** task seed 与 scenario template 的 random intercept, 必要时加入 method random slope;
6. **非劣效检验:** 对 FSS 使用 $\epsilon = 2$ 个百分点的单侧非劣效检验;
7. **多重比较:** 主假设采用 Holm 校正; 探索性分析采用 Benjamini–Hochberg FDR;
8. **效果量:** 同时报 absolute difference、relative reduction、odds ratio 和置信区间, 不只报 p 值。

6.6 预注册成功标准

以下是项目级 go/no-go 标准, 而非先验保证:

表 4: 预注册成功阈值

指标	最低成功	强成功
未缓存总 token	相比 MAS-FullSync 下降 $\geq 20\%$	下降 $\geq 35\%$
Final State Success	非劣于最佳安全基线 2 pp	不下降且显著提高
EvidenceValidRate	$\geq 95\%$	$\geq 97\%$
UAR/CWR	不高于各基线, UAR 相对下降 $\geq 20\%$	UAR/CWR 相对下降 $\geq 25\%$
CWCR	$\geq 80\%$	$\geq 90\%$
Recovery	$\geq 50\%$	$\geq 65\%$
Overblock	$\leq 12\%$	$\leq 8\%$
平均 reconciliation/trial	≤ 1.5	≤ 1.2

7 消融实验、混淆控制与稳健性分析

7.1 核心消融

- 1. **No-MSE**: 所有 agent 接收完整 evidence, 但保留 ledger 与 gate;
- 2. **No-Delta**: 只使用 raw pointer, 不提供字段级变化;
- 3. **No-Provenance**: 移除 source、version、digest 和 dependency links;
- 4. **No-Gate**: worker 可直接执行高风险写;
- 5. **No-Selective-Requery**: 证据无效时只能 block, 不允许恢复;
- 6. **Fixed-Budget**: 所有回合固定 low、normal 或 high reasoning;
- 7. **Oracle-Schema**: 使用人工核验的完整 required evidence schema, 估计 schema 错误造成的上限损失;
- 8. **No-Version-Check**: 保留字段完整性, 但忽略 freshness, 单独测量版本机制贡献。

7.2 主要混淆因素与控制

表 5: 混淆因素、风险与控制方案

混淆因素	可能造成的错误解释	控制方式
模型能力	强模型可能同时更省重试、也更会处理冲突	主要因果比较在同一模型内部; 模型作为固定效应与交互项
架构拓扑	supervisor、pipeline、black-board 的消息结构不同	主实验固定单一拓扑; 跨拓扑仅作为外推分析

混淆因素	可能造成的错误解释	控制方式
Prompt 长度	方法 prompt 本身更长可能抵消 token 节约	所有 system prompt 计入 token；公开模板并报告静态 prompt 长度
摘要质量	LLM 摘要失真可能被误认为 visibility 效应	主方法使用 deterministic header/delta；LLM summarization 仅作为独立基线
环境随机性	库存、价格、用户行为变化可能导致轨迹差异	task seed、mutation seed、user simulator seed 配对固定
解码随机性	相同上下文仍可能产生不同动作	temperature 0、重复运行、配对随机种子、任务级聚类
量化与 serving	不同精度影响模型性能或 tool parsing	方法比较固定 checkpoint/precision/server；跨模型不做直接因果归因
Provider cache	重复上下文可能被缓存，夸大或掩盖算法收益	报 tokenizer-level uncached tokens；cache on/off 做敏感性分析
Gate 额外调用	安全提升可能来自更多 LLM 调用而非证据机制	与等调用预算 verifier baseline 比较；报告 LLM/tool call 数
过度阻断	UAR 下降可能只是因为从不执行写	同时报告 FSS、Overblock、Recovery 和 abstention
Schema 泄漏	action schema 可能间接包含正确答案	schema 只描述字段类型/前置条件，不包含任务具体值；审计模板
任务选择偏差	只选适合方法的长任务会高估收益	预注册筛选规则，公布被排除任务及原因，补充全量 benchmark 分析

7.3 稳健性与负面结果分析

将特别分析以下边界条件：

- 当冲突密度很高、每一步都需要 raw fetch 时，RAVEL 是否失去 token 优势；
- 当任务主要是只读查询、几乎没有状态写时，commit gate 是否只增加成本；
- 当 required evidence schema 不完整时，系统是否产生“看似验证、实际漏证据”的虚假安全；
- 当关键证据存在于自由文本而非结构化字段时，字段级 ledger 是否不足；
- 当不同 agent 的分歧来自政策解释而非事实版本时，版本机制是否无法解决语义冲突；
- 当环境 mutation 发生在 validation 与 commit 之间时，是否需要 compare-and-swap、锁或事务支持。

这些负面结果不会被当作实现失败，而将用于界定 claim：RAVEL 的主要适用范围是具有可识别高风险写、可结构化关键字段、可版本化工具状态和可控制提交入口的 agent 系统。

8 预期结果与论文叙事

8.1 预期但不预设的结果模式

理想结果不是所有指标均单调改善，而是出现清晰的 Pareto 结构：

- MAS-FullSync 具有较高 token 成本，可能在 clean regime 下保持较好效用，但在 conflicting version 下仍可能错误写；
- 纯 observation masking 显著省 token，但在 action-critical 字段被遮蔽时可能提高 UAR；
- Ledger-Only 改善 traceability 与 token，但缺乏 gate 时不能保证写前证据完整；
- Commit-Gate-Only 提高安全，但因为仍使用 full context，token 节约有限；
- RAVEL-Full 在大多数任务上接近或超过安全基线的 FSS，同时显著降低 token，并在 delayed/conflicting regimes 下取得更低 UAR/CWR；
- 在极高冲突密度或极度非结构化任务上，RAVEL 的 token 优势缩小，形成明确适用边界。

8.2 论文主图与主表

建议最终论文至少包含：

1. **Token-Safety Pareto 图**：横轴 uncached token，纵轴 EvidenceValid 或 $1-UAR$ ；
2. **Trajectory Divergence Timeline**：展示同一 seed 在四个 regimes 下何时开始分叉；
3. **Reconciliation Waterfall**：candidate write $\rightarrow$ caught $\rightarrow$ field fetch $\rightarrow$ requery $\rightarrow$ commit/block；
4. **主结果表**：四模型、三域、四 regimes 下的 FSS、token、UAR、Overblock；
5. **组件消融表**：MSE、VDL、gate、ARB 的边际贡献；
6. **Failure case study**：至少各给一个 stale write、conflicting write、schema omission 和 overblock 案例。

8.3 可接受的 claim 边界

若实验支持假设，论文可以主张：

在可版本化工具环境和高风险写任务中，证据可见性是影响多智能体工具轨迹与安全的重要运行时变量；通过外部 ledger、最小充分路由和写前证据验证，可以在保持任务效用非劣的条件下降低上下文 token，并增强对陈旧和冲突证据的鲁棒性。

论文不应直接主张：RAVEL 对所有 multi-agent architecture、所有自由文本工具或真实生产系统提供无条件安全保证；也不应把模拟环境中的 EvidenceValid 等同于现实法律、金融或医疗操作的完整安全性。

9 工程实施计划

9.1 阶段与交付物

表 6: 十二周研究计划

周期	核心工作	交付物
Week 1–2	扫描并复用现有实验资产；固化日志 schema、task seed、四类 visibility wrapper；复现原始 baseline	资产清单、可复现 runner、baseline 报告
Week 3–4	实现 versioned ledger、canonicalization、header/delta/pointer 与 provenance links	Ledger-only baseline、单元测试、数据字典
Week 5–6	建立 high-risk action schema、MSE-Router、候选写结构与 commit gate	RAVEL-MSE 原型、EvidenceValid evaluator
Week 7–8	实现 selective requery、reconciliation ladder、budget adapter；完成四模型 serving 适配	RAVEL-Full、模型适配测试、开发集冻结
Week 9–10	运行 Stage A 与核心 Stage B；完成 paired bootstrap 和 mixed-effects 分析	主表、主图、统计脚本、错误案例库
Week 11	完整消融、cache 敏感性、次级架构与 AgentDojo 可选外推	消融表、外推结果、威胁分析
Week 12	冻结配置、复现实验、撰写论文和 artifact 文档	论文初稿、附录、代码与复现说明

9.2 软件与日志要求

每个 trial 必须保存：

- task ID、domain、model、checkpoint、precision、server config、seed；
- system/user prompts 的哈希与版本；
- 每个 agent 实际可见的 message/evidence IDs，而非只保存全局日志；
- 完整 tool calls、参数、返回值、对象版本和环境 mutation；
- candidate writes、gate verdict、required fields、缺失/冲突原因；
- input/output/cached/uncached tokens、latency、retries；
- final state、official reward、policy violations、所有安全指标；
- 可重放的 ledger snapshot 与 trajectory event log。

日志中的隐式 chain-of-thought 不作为必要 artifact；系统只保存模型公开输出、结构化决策、工具参数和可审计的简短 rationale，避免把不可验证的内部推理当作 provenance。

10 风险、伦理与局限

10.1 方法风险

- **Schema 不完整**：是本方法最重要的系统性风险。将通过 oracle-schema 上限、shadow logging 与人工错误审计量化；
- **语义冲突难以结构化**：对自由文本政策解释和隐含约束，字段级 conflict detection 可能漏报；
- **版本控制依赖后端**：若外部工具不提供版本、时间戳或原子条件写，则 ledger 只能提供审计，不能防止验证后状态再次变化；
- **推理预算接口不一致**：ARB 只能做模型内比较，不应把不同 provider 的“high thinking”视为同一数值处理；
- **benchmark 外推有限**：airline、retail 和 telecom 虽具有真实 API 语义，但仍是模拟环境。

10.2 安全与伦理边界

本研究只在模拟或隔离环境中执行状态写，不连接真实银行、医疗、支付或关键基础设施。高风险写的定义用于评价 agent 系统的程序性证据与状态一致性，不替代现实中的法律授权、身份认证、人类审批或行业合规。若未来部署到真实系统，应将 RAVEL 视为附加控制层，而不是唯一安全边界。

10.3 开放科学与复现

计划公开：任务筛选规则、visibility wrapper、action schemas、ledger schema、评估脚本、统计代码、固定 prompts 与非敏感轨迹。对于受模型许可限制的 checkpoints，仅发布可复现配置和哈希。所有探索性分析与预注册主分析分开标注。

11 结论

本 proposal 将已有的“证据可见性改变工具轨迹”现象提升为一个可形式化、可干预和可优化的研究问题。RAVEL 的关键不在于让 agent 看到更多信息，而在于把证据的完整存储、局部可见、版本冲突、动作依赖和写前验证分层处理：常规回合只提供最小充分证据，高风险写前再按 schema 恢复完整 read-set，并把昂贵的重查与推理预算集中在真正需要的位置。

若实验成功，该工作将给出一种区别于通用 context compression、单纯 fault injection、后端 concurrency control 和工具权限控制的系统路线：**以 evidence visibility policy 为核心变量，在多智能体长程工具任务中联合优化 token、任务效用和写安全**。即使结果不完全支持预期，严格配对的实验仍将回答一个重要基础问题：何种 evidence asymmetry 会导致 trajectory divergence，哪些差异可以通过结构化系统机制修复，哪些则必须依赖更强的语义推理或后端事务保证。

参考文献

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv preprint arXiv:2506.07982*, 2025. doi: 10.48550/arXiv.2506.07982. URL <https://arxiv.org/abs/2506.07982>.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail? In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 38, 2025. URL <https://arxiv.org/abs/2503.13657>.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 37, 2024. URL <https://arxiv.org/abs/2406.13352>.
- Pengfei He, Zhenwei Dai, Bing He, Hui Liu, Xianfeng Tang, Hanqing Lu, Juanhui Li, Jiayuan Ding, Subhabrata Mukherjee, Suhang Wang, Yue Xing, Jiliang Tang, and Benoit Dumoulin. Traject-bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025. URL <https://arxiv.org/abs/2510.04550>.
- Kyochul Jang, Donghyeon Lee, Kyusik Kim, Dongseok Heo, Taewhoo Lee, Woojeong Kim, and Bongwon Suh. Dice-bench: Evaluating the tool-use capabilities of large language models in multi-round, multi-party dialogues. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. URL <https://aclanthology.org/2025.findings-acl.1375/>.
- Jin Jia, Zhiling Deng, Zhuangbin Chen, Yingqi Wang, and Zibin Zheng. Mas-fire: Fault injection and reliability evaluation for llm-based multi-agent systems. *arXiv preprint arXiv:2602.19843*, 2026. URL <https://arxiv.org/abs/2602.19843>.
- Sajjad Khan. S-bus: Automatic read-set reconstruction for multi-agent llm state coordination. *arXiv preprint arXiv:2605.17076*, 2026. doi: 10.48550/arXiv.2605.17076. URL <https://arxiv.org/abs/2605.17076>.
- Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management. *arXiv preprint arXiv:2508.21433*, 2025. doi: 10.48550/arXiv.2508.21433. URL <https://arxiv.org/abs/2508.21433>.
- Meta AI. Llama-3.3-70b-instruct model card. Hugging Face Model Card, 2024. URL <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>. Accessed 2026-06-14.
- Yansong Ning, Jun Fang, Naiqiang Tan, and Hao Liu. Agent-omit: Training efficient llm agents for adaptive thought and observation omission via agentic reinforcement learning. *arXiv preprint arXiv:2602.04284*, 2026. URL <https://arxiv.org/abs/2602.04284>.

- OpenAI. gpt-oss-120b & gpt-oss-20b model card. OpenAI Model Card, 2025. URL <https://openai.com/index/gpt-oss-model-card/>. Accessed 2026-06-14.
- Qwen Team. Qwen3-32b model card. Hugging Face Model Card, 2025. URL <https://huggingface.co/Qwen/Qwen3-32B>. Accessed 2026-06-14.
- Rohan Sequeira, Stavros Damianakis, Umar Iqbal, and Konstantinos Psounis. Agent-sentry: Bounding llm agents via execution provenance. *arXiv preprint arXiv:2603.22868*, 2026. URL <https://arxiv.org/abs/2603.22868>.
- Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for llm agents. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- Yiqi Wang, Jiaqi Zhang, Taotao Cai, Zirui Liu, Qingqiang Sun, Zequn Sun, Zhangkai Wu, Mingkai Zhang, and Yanming Zhu. From agent traces to trust: Evidence tracing and execution provenance in llm agents. *arXiv preprint arXiv:2606.04990*, 2026. URL <https://arxiv.org/abs/2606.04990>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- Yating Wu, Yuhao Zhang, Sayan Ghosh, Sourya Basu, Anoop Deoras, Jun Huan, and Gaurav Gupta. Contextweaver: Selective and dependency-structured memory construction for llm agents. *arXiv preprint arXiv:2604.23069*, 2026. doi: 10.48550/arXiv.2604.23069. URL <https://arxiv.org/abs/2604.23069>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*, 2024. doi: 10.48550/arXiv.2406.12045. URL <https://arxiv.org/abs/2406.12045>.
- Z.ai. Glm-4.5-air model card. Hugging Face Model Card, 2025. URL <https://huggingface.co/zai-org/GLM-4.5-Air>. Accessed 2026-06-14.